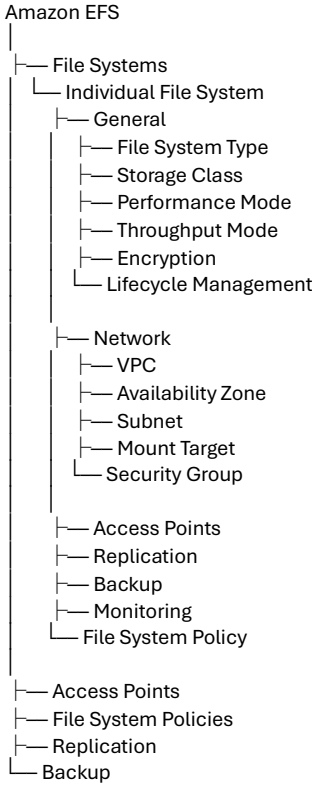
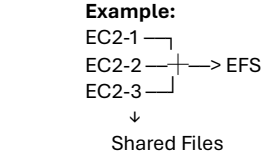
Amazon EFS — Console Dashboard Wise Important Interview Questions & Answers

EFS Console Structure



---

1. What is Amazon EFS?
Answer:
Amazon EFS stands for Elastic File System and provides managed shared file storage in AWS.
Multiple EC2 instances can access the same files at the same time using the NFS protocol.
EFS automatically grows and shrinks based on the amount of data stored.
It is mainly used when multiple servers need common shared storage.
For example:
Suppose we have multiple EC2 servers running the same application.
All EC2 servers need to access the same uploaded files.
We can store those files in EFS and mount EFS on all EC2 servers.
Example:



Simple ga gurthupettuko:
EFS = Shared File Storage for multiple servers

EFS → File Systems
2. What is a File System in EFS?
Answer:
A File System is the main storage resource created in Amazon EFS.
It is where our application files and directories are actually stored.
We can configure its performance, throughput, encryption, lifecycle and network settings.
EC2 instances access this file system through EFS mount targets.
For example:
Suppose our application needs a shared folder called /shared-data.
We create an EFS file system and mount it on multiple EC2 instances.
All those instances can then read and write the same files.
Example:



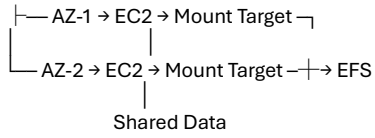
Simple ga gurthupettuko:
File System = Main storage in EFS

---

3. What is the difference between EFS and EBS?
Answer:
EBS is normally attached to an EC2 instance as block storage, while EFS is shared file storage.
One EFS file system can be accessed by multiple EC2 instances at the same time.
EBS is commonly used for operating systems, applications and databases.
EFS is commonly used for shared files and directories.
For example:
Suppose we have one EC2 server running a database.
We can use EBS for the database storage.
If multiple EC2 servers need the same files, we can use EFS.
Example:
EBS → EC2 → Block Storage

EFS → EC2-1
→ EC2-2
→ EC2-3
→ Shared Storage
Simple ga gurthupettuko:
EBS = Block Storage | EFS = Shared File Storage

---

4. What is a Regional EFS File System?
Answer:
A Regional EFS file system stores data redundantly across multiple Availability Zones in an AWS Region.
It is designed for high availability and durability.
We can create mount targets in multiple Availability Zones for access from different subnets.
Regional is the recommended choice for most production workloads.
For example:
Suppose our application runs in two Availability Zones.
We create a Regional EFS file system and mount it through mount targets in both AZs.
Both application servers can access the same EFS data.
Example:
Region


Simple ga gurthupettuko:
Regional EFS = Multi-AZ shared file storage

---

5. What is a One Zone EFS File System?
Answer:
One Zone EFS stores its data within a single Availability Zone.
It is generally lower cost than Regional EFS but does not provide the same multi-AZ resilience.
A One Zone file system can have a mount target only in its Availability Zone.
It is useful when the workload can tolerate AZ-level risk or when cost optimization is important.
For example:
Suppose an application is designed to run only in one Availability Zone.
We can use One Zone EFS to store its files at a lower cost.
For critical production data, Regional EFS is generally preferred.
Example:
AWS Region
↓
AZ-1
↓
One Zone EFS
↓
Application Files
Simple ga gurthupettuko:
One Zone = One AZ | Regional = Multiple AZs

File System → Storage / Performance
6. What are EFS Storage Classes?
Answer:
EFS provides different storage classes based on how frequently files are accessed.
EFS Standard is designed for frequently accessed files with low latency.
EFS Infrequent Access and EFS Archive are designed for less frequently accessed data.
Lifecycle policies can automatically move files between suitable storage classes.

For example:
Suppose application files are accessed every day.
We can keep frequently accessed files in EFS Standard.
Older files that are rarely accessed can automatically move to IA or Archive.
Example:
Frequently Used
↓
EFS Standard

Rarely Used
↓
EFS IA / Archive
Simple ga gurthupettuko:
Storage Class = Choose storage based on access frequency

---

7. What is EFS Lifecycle Management?
Answer:
Lifecycle Management automatically moves files between EFS storage classes based on access patterns.
It helps reduce storage cost for files that are not accessed frequently.
We can configure transition policies for moving files to EFS IA or Archive.
This happens automatically without manually moving individual files.
For example:
Suppose old application reports are rarely accessed after some time.
We can configure a lifecycle policy to move those files to a lower-cost storage class.
Frequently accessed files can remain in Standard storage.
Example:
New File
↓
EFS Standard
↓
Not Frequently Accessed
↓
EFS IA / Archive
Simple ga gurthupettuko:
Lifecycle = Automatically move old/rarely used files

---

8. What is EFS Performance Mode?
Answer:
Performance Mode controls the performance characteristics of an EFS file system.
EFS provides General Purpose and Max I/O performance modes.
General Purpose provides lower per-operation latency and is recommended for most workloads.
Max I/O is an older option designed for highly parallel workloads that can tolerate higher latency.
For example:
Suppose our web application needs fast file operations.
We normally choose General Purpose performance mode.
We use Max I/O only when the workload specifically requires highly parallel access and can tolerate higher latency.
Example:
EFS
↓
Performance Mode
├— General Purpose → Lower Latency
└— Max I/O → Higher Parallelism / Higher Latency
Simple ga gurthupettuko:
Performance Mode = How EFS handles file-operation performance


A AWS Documentation+1

---

9. What is EFS Throughput Mode?
Answer:
Throughput Mode controls how much throughput the EFS file system can provide.
EFS currently provides Elastic, Provisioned and Bursting throughput modes.
Elastic automatically adjusts throughput based on workload activity and is the default recommended mode.
Provisioned lets us specify throughput, while Bursting scales with the amount of stored data.
For example:
Suppose our application has unpredictable traffic and file activity.
Elastic throughput can automatically adjust to the workload.
If we know the required throughput in advance, Provisioned throughput may be useful.
Example:
EFS Throughput

├— Elastic → Automatically scales
├— Provisioned → We specify throughput
└— Bursting → Based on storage size

Simple ga gurthupettuko:

Throughput Mode = How much file traffic EFS can handle



A AWS Documentation+1

---

10. What is the difference between Performance Mode and Throughput Mode?

Answer:

Performance Mode mainly controls the latency and behavior of file operations.
Throughput Mode controls how much data can be transferred through the file system.
Performance Mode answers “how quickly individual operations behave,” while Throughput Mode answers “how much data traffic can be handled.”
Both settings affect EFS performance but solve different requirements.

For example:

Suppose an application needs low latency for file operations.
We select an appropriate Performance Mode.
If the application also needs high data transfer capacity, we choose a suitable Throughput Mode.

Example:

EFS Performance
├— Performance Mode → Operation Latency
└— Throughput Mode → Data Transfer Capacity

Simple ga gurthupettuko:

Performance = Latency | Throughput = Data Capacity

File System → Encryption

11. What is EFS Encryption at Rest?

Answer:

Encryption at rest protects data stored inside the EFS file system.
EFS can use AWS KMS keys to encrypt the data and metadata stored on the file system.
Encryption can be enabled when creating the file system.
The encryption setting cannot be changed on an existing file system.

For example:

Suppose we store confidential company documents in EFS.
We enable encryption when creating the EFS file system.
The stored data is then protected using encryption.

Example:

Application
↓
EFS
↓
Encrypted Data
↓
Storage

Simple ga gurthupettuko:

Encryption at Rest = Protect stored EFS data



A AWS Documentation

---

12. What is EFS Encryption in Transit?

Answer:

Encryption in transit protects data while it is moving between an EC2 client and EFS.
EFS uses TLS to encrypt the connection when mounting the file system with the appropriate mount option.
This protects file data while travelling through the network.
File system policies can also be used to enforce encrypted access.

For example:

Suppose an EC2 server needs to access confidential files from EFS.
We mount EFS using TLS so the communication between EC2 and EFS is encrypted.
This protects the data while it travels across the network.

Example:

EC2
↓
TLS / Encrypted Connection
↓
EFS
↓

Files
Simple ga gurthupettuko:
At Rest = Stored Data | In Transit = Moving Data

A AWS Documentation

File System → Network
13. What is an EFS Mount Target?
Answer:
A mount target provides network access to an EFS file system from a VPC.
EC2 instances use the mount target to connect to and mount the EFS file system.
A mount target is associated with a subnet, Availability Zone, security groups and an IP address.
For Regional EFS, we normally create mount targets in the Availability Zones where applications need access.
For example:
Suppose EC2 is running in two Availability Zones.
We create an EFS mount target in each required AZ and subnet.
The EC2 instances then connect to EFS through those mount targets.
Example:
EC2-AZ1
↓
Mount Target-AZ1
↓
EFS
↑
Mount Target-AZ2
↑
EC2-AZ2
Simple ga gurthupettuko:
Mount Target = Network entry point to EFS

A AWS Documentation+1

---

14. Why do we create EFS Mount Targets in multiple Availability Zones?
Answer:
Mount targets provide network access to EFS from specific Availability Zones and subnets.
Creating them in multiple AZs allows applications in those AZs to access EFS locally through the VPC network.
This supports high availability and avoids depending on a single AZ for network access.
For Regional EFS, this is a common production design.
For example:
Suppose our application has EC2 instances in AZ-1 and AZ-2.
We create mount targets in both AZs.
Both EC2 groups can access the same EFS file system.
Example:
AZ-1 → EC2 → Mount Target
↓
EFS
↑
AZ-2 → EC2 → Mount Target
Simple ga gurthupettuko:
Multiple AZs = Multiple EFS network access points

---

15. What is the role of a Security Group with EFS?
Answer:
A security group controls network traffic to an EFS mount target.
EFS uses the NFS protocol, which normally communicates through TCP port 2049.
The EFS security group should allow NFS traffic from the security group of the application EC2 instances.
This allows only the required servers to connect to the file system.
For example:
Suppose EC2 servers use EC2-SG.
We attach EFS-SG to the EFS mount target and allow TCP 2049 from EC2-SG.
Now the application servers can mount EFS.
Example:
EC2
↓
EC2 Security Group
↓
TCP 2049

↓
EFS Security Group
↓
EFS
Simple ga gurthupettuko:
EFS NFS = TCP 2049

---

16. What is the VPC requirement for EFS?
Answer:
EFS integrates with a VPC through mount targets.
A file system can have mount targets in the VPC where applications need network access.
The mount target is placed inside a subnet and protected using security groups.
EC2 instances in the VPC can then mount the EFS file system.
For example:
Suppose our EC2 servers are inside a custom VPC.
We create EFS mount targets in suitable subnets of that VPC.
The EC2 instances can then connect to EFS through the VPC network.
Example:
VPC
│
├── Subnet-A → EC2
│ ↓
│ EFS Mount Target
│
└── Subnet-B → EC2
↓
EFS Mount Target
Simple ga gurthupettuko:
EFS + VPC = Mount Targets + Subnets + Security Groups

EFS → Access Points
17. What is an EFS Access Point?
Answer:
An EFS Access Point provides an application-specific entry point into an EFS file system.
It can define a root directory and enforce a POSIX user and group identity for clients.
Different applications can use different access points while sharing the same EFS file system.
Access points help simplify and control application-level access to shared storage.
For example:
Suppose one EFS contains data for two applications.
Application A should access /app-a and Application B should access /app-b.
We can create separate access points for these applications.
Example:
EFS
│
┌──────┴──────┐
↓ ↓
Access Point A Access Point B
↓ ↓
/app-a /app-b
↓ ↓
Application A Application B
Simple ga gurthupettuko:
Access Point = Controlled application entry into EFS


A AWS Documentation

---

18. Why do we use EFS Access Points?
Answer:
Access Points are useful when multiple applications need different access to the same EFS file system.
They can enforce a specific POSIX identity and root directory for an application.
This prevents applications from directly using the entire file-system namespace.
They are especially useful for multi-tenant or application-specific storage designs.
For example:
Suppose a company has many applications using one EFS.
Instead of giving every application access to the whole file system, we create separate access points.
Each application can then work with its assigned directory.
Example:
One EFS
│

├— Access Point A → /app-a
├— Access Point B → /app-b
└— Access Point C → /app-c

Simple ga gurthupettuko:
Access Points = Separate controlled paths for applications

EFS → File System Policy
19. What is an EFS File System Policy?
Answer:
An EFS File System Policy is a resource-based policy attached to an EFS file system.
It controls how clients can access the file system and can define conditions for allowed actions.
It can be used to prevent root access, require encrypted connections, or prevent anonymous access.
It works together with IAM and network controls.
For example:
Suppose we want all EFS clients to use encrypted connections.
We can configure the file system policy to deny access from clients that do not use transit encryption.
This adds another layer of security to EFS access.
Example:
Client
↓
EFS File System Policy
↓
Security Rules
↓
Allow / Deny
↓
EFS
Simple ga gurthupettuko:
File System Policy = Control how clients access EFS


A AWS Documentation

---

20. What are the important EFS File System Policy options?
Answer:
EFS provides policy options such as preventing root access by default.
We can also enforce read-only access and prevent anonymous access.
Another important option is enforcing in-transit encryption for clients.
These options help create stronger controls for EFS access.
For example:
Suppose a production EFS contains sensitive files.
We can prevent anonymous access and require encrypted connections.
We can also prevent clients from using root-level access.
Example:
EFS Policy
├— Prevent Root Access
├— Read-Only Access
├— Prevent Anonymous Access
└— Require In-Transit Encryption
Simple ga gurthupettuko:
EFS Policy = Root + Read/Write + Anonymous + Encryption controls


A AWS Documentation

EFS → Replication
21. What is EFS Replication?
Answer:
EFS Replication automatically copies data from a source EFS file system to a destination EFS file system.
It is mainly used for disaster recovery and maintaining another copy of important file data.
The destination can be in another AWS Region for cross-Region disaster recovery.
Replication is managed by AWS after the configuration is created.
For example:
Suppose our production EFS is in the Mumbai Region.
We can replicate its data to another EFS file system in another Region.
If a major regional problem occurs, the replicated data provides another recovery option.
Example:
Primary Region
↓

Source EFS
↓
EFS Replication
↓
Destination EFS
↓
Backup Region
Simple ga gurthupettuko:
EFS Replication = Copy EFS data for recovery

EFS → Backup
22. What is EFS Backup?
Answer:
EFS can be protected using AWS Backup for centralized backup management.
AWS Backup can create and manage backups according to backup plans and retention requirements.
Backups provide a recovery option if files are accidentally deleted or the file system needs to be restored.
This is different from replication because backup is designed for recovery points rather than continuous file-system replication.
For example:
Suppose an application stores important documents in EFS.
We configure AWS Backup to periodically back up the EFS file system.
If files are accidentally deleted, we can use the backup for recovery.
Example:
EFS
↓
AWS Backup
↓
Backup
↓
Recovery
↓
Restored EFS
Simple ga gurthupettuko:
Backup = Recovery copy of EFS data

---

23. What is the difference between EFS Replication and EFS Backup?
Answer:
EFS Replication maintains a replicated copy of data in another file system for disaster recovery.
AWS Backup creates recovery points that can be used to restore data.
Replication is mainly useful for maintaining another operational copy, while backup is useful for point-in-time recovery.
For important production systems, both can be used for different recovery requirements.
For example:
Suppose our production EFS is accidentally damaged or a Region becomes unavailable.
Replication can provide another copy in a different location.
A backup can help restore the data from an appropriate recovery point.
Example:
EFS
├── Replication → Another EFS
│ ↓
│ DR Copy
│
└── Backup → Recovery Point
↓
Restore
Simple ga gurthupettuko:
Replication = Another copy | Backup = Recovery point

EFS → Monitoring
24. How do we monitor EFS?
Answer:
Amazon EFS provides monitoring metrics through Amazon CloudWatch.
We can monitor file-system activity, throughput usage and other performance-related metrics.
CloudWatch alarms can be configured when important metrics cross defined thresholds.
Monitoring helps identify performance, usage and operational problems.
For example:
Suppose our application suddenly starts using a large amount of EFS throughput.
We can monitor EFS metrics in CloudWatch and create an alarm for abnormal usage.
This helps us investigate the workload before it becomes a bigger problem.
Example:
EFS
↓
CloudWatch Metrics

↓
Alarm
↓
High Usage
↓
Investigation
Simple ga gurthupettuko:
EFS Monitoring = CloudWatch Metrics + Alarms


AWS Documentation+1

---

25. What is the EFS DNS name used for mounting?
Answer:
Each EFS file system has a DNS name that clients can use when mounting the file system.
The DNS name is associated with the EFS file system and AWS Region.
When an EC2 instance mounts EFS, the EFS mount helper can use this DNS name.
The instance must have the required network connectivity and mount target available.
For example:
Suppose our EFS file system ID is fs-12345678.
An EC2 instance can use the EFS DNS name to mount the file system.
After mounting, the EFS files appear as a normal directory on the EC2 server.
Example:
EC2
↓
EFS DNS Name
↓
Mount Target
↓
EFS
↓
/shared
Simple ga gurthupettuko:
EFS DNS = Address used by clients to reach EFS


AWS Documentation

EFS → Mounting
26. How do we mount EFS on an EC2 instance?
Answer:
First, the EC2 instance must have network connectivity to an EFS mount target.
We normally install and use the Amazon EFS mount helper on the EC2 instance.
Then we mount the EFS file system to a directory such as /mnt/efs.
After mounting, the application can read and write files using that directory.
For example:
Suppose our EC2 server needs a shared directory for uploaded files.
We create the EFS mount target and configure the security group for NFS traffic.
Then we mount EFS on /mnt/efs and store application files there.
Example:
EC2
↓
EFS Mount Helper
↓
Mount Target
↓
EFS
↓
/mnt/efs
Simple ga gurthupettuko:
Mount EFS = Connect EFS to an EC2 directory

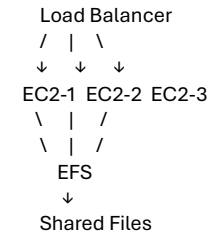
Important Interview Differences
27. What is the difference between EFS and S3?
Answer:
EFS is a managed file system that provides normal file-system access through NFS.
S3 is object storage where data is stored as objects inside buckets.
Applications use EFS when they need shared file-system access and S3 when they need scalable object storage.
EFS is commonly mounted by servers, while S3 is accessed through APIs, SDKs or URLs.

For example:
Suppose multiple EC2 servers need to share the same application files.
We can use EFS because all servers can mount it.
For images, backups and documents that are stored as objects, S3 is often a better choice.
Example:
EFS → File System → Mount → EC2

S3 → Bucket → Object → Application/API
Simple ga gurthupettuko:
EFS = Files | S3 = Objects

---

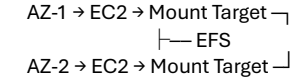
28. Can multiple EC2 instances use the same EFS?
Answer:
Yes, multiple EC2 instances can mount and access the same EFS file system.
This is one of the main reasons EFS is used for shared storage.
Instances can read and write common files according to their file-system permissions and access controls.
The EC2 instances can be in different Availability Zones within the Region.
For example:
Suppose we have three EC2 instances running behind an Application Load Balancer.
All three servers need access to the same uploaded images.
We can mount the same EFS file system on all three EC2 instances.
Example:



Simple ga gurthupettuko:
One EFS → Many EC2 → Same Files

---

29. Can EFS be used with EC2 in different Availability Zones?
Answer:
Yes, Regional EFS can be accessed by EC2 instances running in different Availability Zones.
We normally create mount targets in the required Availability Zones.
Each EC2 instance connects through the appropriate mount target using the VPC network.
This makes EFS suitable for highly available applications running across multiple AZs.
For example:
Suppose our application has EC2 instances in AZ-1 and AZ-2.
We create EFS mount targets in both AZs.
Both application servers access the same EFS data.
Example:



Simple ga gurthupettuko:
Regional EFS = Same shared files across AZs
